

AVE Cloud Skills 深度集成文档 — AiTrading Pro

基于 AVE Cloud Skills 构建的 AI 多链加密货币自动交易系统

版本: 2.0 | 更新日期: 2026-04-10

目录

1. [项目概述](#1-项目概述)
2. [使用的 AVE Skills](#2-使用的-ave-skills)
3. [模块一：热币发现引擎](#3-模块一热币发现引擎)
4. [模块二：安全检测系统](#4-模块二安全检测系统)
5. [模块三：实时价格基础设施](#5-模块三实时价格基础设施)
6. [模块四：聪明钱情报网络](#6-模块四聪明钱情报网络)
7. [模块五：交易执行层](#7-模块五交易执行层)
8. [系统架构全景图](#8-系统架构全景图)
9. [API 调用明细表](#9-api-调用明细表)
10. [核心实现 ave_client.py](#10-核心实现-ave_clientpy)
11. [AI 决策引擎概览](#11-ai-决策引擎概览)
12. [效果指标与运营数据](#12-效果指标与运营数据)
13. [链支持矩阵](#13-链支持矩阵)
14. [技术创新点](#14-技术创新点)
15. [环境配置](#15-环境配置)
16. [License](#16-license)

1. 项目概述

AiTrading Pro 是基于 AVE Cloud Skills 构建的 AI

多链加密货币自动交易系统。系统覆盖从信号发现到链上执行的完整闭环，横跨

Solana、BSC、Ethereum、Base 四条公链，通过五大独立信号源（热币趋势、pump.fun

内盘、聪明钱跟踪、KOL 舆情、BTC/ETH 大盘）驱动三层 AI 决策引擎（规则过滤 + Claude 快评 +

多角色辩论），最终经由 AVE 链上交易基础设施完成自托管 DEX 交易。

AVE 在系统中的定位

AVE Cloud Skills

是系统的数据与交易基础设施。从代币发现、实时价格获取、合约安全审计、聪明钱情报到最终的链上 Swap 执行，所有与链上世界的交互均通过 AVE 的两大 Skill 完成：

- ave-data-rest：多链代币数据聚合，提供趋势发现、实时报价、合约风险扫描、聪明钱地址库
- ave-trade-chain-wallet：自托管交易执行，支持报价查询、交易构造、签名提交的完整生命周期

整套系统实现了「AVE 提供链上基础设施 + AI 提供策略智能 + 本地自托管保障资金安全」的三位一体架构。

核心能力

- 多链覆盖：Solana / BSC / ETH / Base 四链并行监控
- 五源信号：热币榜单、pump.fun 内盘、聪明钱追踪、KOL 舆情、BTC/ETH 宏观
- 三级 AI 决策：L1 规则引擎快筛 → L2 Claude LLM 深度分析 → L3 牛熊辩论置信度评分
- 三层记忆系统：工作记忆（当前持仓上下文）→ 情景记忆（历史交易经验）→ 语义记忆（市场模式知识库）
- 七状态市场适应：Regime Detector 实时识别 BULL / BEAR / SIDEWAYS / VOLATILE / CRISIS / RECOVERY / ACCUMULATION
- 自托管交易：私钥永远在本地，AVE 只负责路由和构建交易数据

技术栈

2. 使用的 AVE Skills

2.1 ave-data-rest

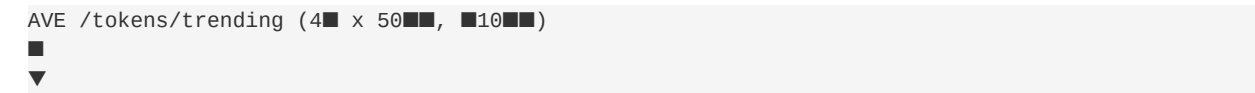
提供多链代币数据聚合能力，是系统的数据心脏。

2.2 ave-trade-chain-wallet

提供自托管多链交易能力，是系统的执行手臂。

3. 模块一：热币发现引擎

数据流全景




```

buy_ratio = buys_1h / (buys_1h + sells_1h)
ratio >= 75% → 10.0
50% ~ 75% → linear(ratio, 0.5, 0.75) * 10
ratio < 50% → 0.0
1h 24h 7

```

M5. 动量新鲜度 (8分)

```

1h > 24h → 
1h << 24h → 

freshness = pc_1h / (pc_24h / 24)
freshness >= 5 → 8.0 # 
freshness 2~5 → 4~8 # 
freshness 1~2 → 0~4 # 
freshness < 1 → # 

```

Q 品质维度 (满分 30)

Q1. 持有者数量 (10分)

```

holders >= 1000 → 10.0
holders 150~1000 → linear 
holders < 150 → 3.0 ( )

```

Q2. 社交存在性 (6分)

```

Twitter → 3
Telegram → 2
Website → 1
AVE appendix 

```

Q3. 安全检测 (8分)

```

8.0
goplus_risk → -4.0
→ -2.0
buy_tax > 5% → -2.0
sell_tax > 5% → -2.0
AVE /contracts/ API

```

Q4. 持仓分散度 (6分)

```

top10_holder_pct Top 10 
→ 
linear(1 - top10_pct, 0.2, 0.8) * 6

```

P 潜力维度 (满分 20)

P1. 市值位置 (10分)

```

MC <= $200K → 10.0
MC >= $50M → 0.0
log_ratio = log10(MC / 200K) / log10(50M / 200K)
score = (1 - log_ratio) * 10

```

P2. 代币年龄 (4分)

```

7~30 → 4.0
< 7 → linear(age, 3, 7) * 4
> 30 → 0

```

P3. 多时间帧共振 (6分)

```
timeframe_hits = ██████████ toplist (5m/1h/4h/24h)
4 ██████████ → 6.0 # ██████
3 █ → 4.0
2 █ → 2.0
1 █ → 0.0
```

推荐等级

退出机制（5 条规则）

入榜后的代币持续受 PriceFeed 毫秒级监控，任一条件触发即退出：

退出后继续追踪 3 天，评估退出时机是否合理，数据回馈优化系统。

4. 模块二：安全检测系统

AVE /contracts/ 返回字段

每个候选代币入榜前必须通过安全检测。系统调用 AVE /contracts/{address}-{chain} 获取合约审计数据。

格式转换：AVE → 内部 goplus_risk 格式

系统通过 `_convert_risk_to_goplus()` 方法将 AVE 原始数据转换为内部统一格式：

[illegible]

安全检测逻辑

安全检测在两个阶段执行：

阶段 1：硬过滤（入榜前）

[illegible]

阶段 2：打分减分（Q3 维度）

- `goplus_risk` 标记 → -4 分
- `is_open_source == False` → -2 分
- `buy_tax > 5%` 或 `sell_tax > 5%` → -2 分

阶段 3：交易前二次校验

- Agent 决策买入前，再次调用 /contracts/ 确认安全状态未变化
- 如果检测到新增风险标记，取消交易

持仓分析

holders_detail 数组提供 Top N 持仓地址和占比。系统计算：

- Top 10 集中度：前 10 地址持仓百分比之和，用于 Q4 持仓分散度评分
- Top 1 集中度：最大单一持仓占比，极端集中时触发额外风控警告

— — —

5. 模块三：实时价格基础设施

PriceFeed 架构

PriceFeed 是全局价格缓存管理器。AVE /tokens/{addr}-{chain} 是代币价格的核心数据源，每 2 秒轮询一次所有注册代币的价格。

```

■ PriceFeed ■ ■
■ ① Binance bookTicker WS → SOL/ETH/BNB/BTC ~10-100ms ■
■ ② Helius logsSubscribe → Solana ■ swap ~400ms ■
■ ③ AVE /tokens/ ■ → ■ ~2s ■
■ update_price(addr, price)
▼
■
■
■ ① HotCoinManager ■ → ■ → ■
■ ② SimTrader ■ → TP/SL ■ (15% ■)
■ ③ PerformanceTracker ■ → D0-D30 ■

```

AVE 价格轮询实现

PriceFeed 的 `_poll_all_prices()` 方法每 2 秒轮询一次所有注册代币的价格：

```

async def _poll_ave_prices(self, addr: list, dex_chain: str) -> None:
    from ave_client import ave
    ave_chain = {"solana": "solana", "ethereum": "eth",
                 "bsc": "bsc", "base": "base"}.get(dex_chain, dex_chain)
    for addr in addr:
        detail = await ave.get_token_detail(addr, ave_chain)
        if detail:
            price_str = detail.get("current_price_usd")
            if price_str:
                self.update_price(addr, float(price_str))

```

引用计数架构

PriceFeed 使用引用计数管理代币的生命周期追踪，确保资源不浪费：

```

■■■■BONK ■■■ HotCoinManager ■ SimTrader ■■

1. HotCoinManager ■■ BONK:
price_feed.register_token("BONK_addr", "solana", source="hot_coin")
→ refcount["BONK"] = 1

2. SimTrader ■■ BONK:
price_feed.register_token("BONK_addr", "solana", source="sim_trader")
→ refcount["BONK"] = 2

3. HotCoinManager ■■ BONK:
price_feed.unregister_token("BONK_addr", source="hot_coin")
→ refcount["BONK"] = 1, ■■■■■SimTrader ■■■■

4. SimTrader ■■ BONK:
price_feed.unregister_token("BONK_addr", source="sim_trader")
→ refcount["BONK"] = 0, ■■■■■■■ API ■■

```

价格变更回调

价格变动超过 0.01% 才触发回调，避免噪音：

```
if abs(price - old) / max(old, 1e-12) > 0.0001:
    for cb in self._callbacks:
        cb(addr, price)
```

三个回调消费者各司其职：

— — —

6. 模块四：聪明钱情报网络

AVE /address/smart_wallet/list

系统调用 AVE 聪明钱 API 获取链上高利润钱包地址：

```
async def get_smart_wallets(self, chain: str, limit: int = 100) -> List[dict]:
    data = await self._data_get(
        "/address/smart_wallet/list", {"chain": c, "limit": limit}
    )
```

与内部钱包库的融合

系统维护 15,000+ 聪明钱地址 (SOL ~5,222 + EVM ~10,540)，来自三个供给层：

[illegible]

Heat Score 公式

聪明钱信号的热度评分综合以下因子：

```
heat_score = w1 * wallet_score # ██████████
+ w2 * position_size # █████
+ w3 * timing_score # ████████████
+ w4 * cluster_count # ████████████████

tier_weight:
```

```

elite (score >= 75): ■■ 3.0
verified (score >= 55): ■■ 2.0
watching (score >= 35): ■■ 1.0
blacklisted (score < 30): ■■ 0 (■■■■■■)

concentration_bonus:
3+ ■■■■■■■■ 1h ■■■■■■■■ → +20 bonus
5+ ■■■■■■■■ 1h ■■■■■■■■ → +50 bonus

```

v3 五维度评估体系

每个聪明钱地址按以下五个维度打分，总分 100：

等级划分

实时监控

- SOL 链：Helius WebSocket logsSubscribe 监听 DEX 程序，~400ms 延迟
- EVM 链：公共 WebSocket RPC 订阅 Swap 事件 topic，~2-12s（取决于出块时间）
- 评估周期：每 2 小时重新计算五维度评分
- 降级规则：14 天无交易降级，28 天无交易移除
- Bot 检测：交易间隔 < 1s 且模式规律 → 自动 blacklist

■■■■■

7. 模块五：交易执行层

完整交易流程

AVE Chain Wallet API 提供自托管交易能力。整个流程分为四步：

[illegible]

Step 1: 报价查询

```
POST chainWallet/getAmountOut
■■:
chainIndex: 501 (SOL) / 56 (BSC) / 1 (ETH) / 8453 (Base)
inTokenAddress: SOL / USDT ■■■■■■■■
outTokenAddress: ■■■■■■
amount: ■■■■ (■■■■)
swapType: "buy" / "sell"
```

```

■■:
estimateOut: ■■■■■■
decimals: ■■■■

■■■■:
■■■■■ = |actualOut - estimateOut| / estimateOut
■■■ > ■■ → ■■■■

```

Step 2: 交易构造

```

Solana: POST chainWallet/createSolanaTx
EVM: POST chainWallet/createEvmTx

■■ (Solana):
inTokenAddress, outTokenAddress, amount
walletAddress: ■■■■■■
slippage: ■■■■■ (■■■)
requestTxId: ■■■■ ID
fee: ■■■ (0.0005 SOL)

■■ (EVM):
chainIndex, inTokenAddress, outTokenAddress, amount
walletAddress, slippage

■■:
rawTransaction: ■■■■■■■■■■
requestTxId: ■■■■■■■■■■ ID

```

Step 3: 本地签名（自托管核心）

```

■■■■■■■■■■■■■■■■■■■■ AVE ■■■■■■■■

Solana: ■■ solders ■■■■
keypair = Keypair.from_seed(private_key_bytes)
signed_tx = Transaction.sign(raw_tx, keypair)

EVM: ■■ eth_account ■■■■
account = Account.from_key(private_key_hex)
signed_tx = account.sign_transaction(raw_tx)

```

Step 4: 签名交易广播

```

Solana: POST chainWallet/sendSignedSolanaTx
EVM: POST chainWallet/sendSignedEvmTx

■■:
requestTxId: ■■ Step 2 ■■■■ ID
signedTransaction: Step 3 ■■■■■■■■
chainIndex (EVM): ■■■■

■■:
txHash: ■■■■■■

■■■■:
■■ txHash ■ DB → ■■■■■■ → ■■■■■■

```

Solana 交易代码示例

```
# 1. ■■■
quote = await ave.get_quote("solana", "sol", token_addr, amount, "buy")
estimate_out = quote["estimateOut"]

# 2. ■■■■
tx_data = await ave.create_solana_tx(
    in_token="sol",
    out_token=token_addr,
    amount=amount_raw,
    swap_type="buy",
    slippage=1.0,
    wallet_address=wallet_addr,
    fee=0.0005, # SOL ■■■
)
request_tx_id = tx_data["requestTxId"]
raw_tx = tx_data["rawTransaction"]

# 3. ■■■■■solders ■■
signed = sign_solana_tx(raw_tx, private_key)

# 4. ■■
result = await ave.send_signed_solana_tx(request_tx_id, signed)
tx_hash = result["txHash"]
```

EVM 交易代码示例

```
# 1. ■■
quote = await ave.get_quote("bsc", usdt_addr, token_addr, amount, "buy")

# 2. ■■■■
tx_data = await ave.create_evm_tx(
chain="bsc",
in_token=usdt_addr,
out_token=token_addr,
amount=amount_raw,
swap_type="buy",
slippage=1.0,
wallet_address=wallet_addr,
)
request_tx_id = tx_data["requestTxId"]

# 3. ■■■■■eth-account ■■
signed = sign_evm_tx(tx_data, private_key, chain_id=56)

# 4. ■■
result = await ave.send_signed_evm_tx("bsc", request_tx_id, signed)
tx_hash = result["txHash"]
```

Solana 与 EVM 的关键差异

自托管安全保证

[illegible]

AveClient 类结构

```
class AveClient:
    """AVE Cloud API 1 RPS """
    _session: aiohttp.ClientSession # HTTP
    _last_req_ts: float #
    _lock: asyncio.Lock #
```

公开方法

限速机制

AVE Free Plan 限制 1 RPS。客户端通过异步锁实现精确限速：

```
async def _throttle(self) -> None:
    async with self._lock:
        now = time.time()
        gap = now - self._last_req_ts
        if gap < 1.0:
            await asyncio.sleep(1.0 - gap)
        self._last_req_ts = time.time()
```

- 使用 asyncio.Lock 保证并发安全
- 计算距上次请求的间隔，不足 1 秒则等待
- 所有 _data_get 和 _trade_post 调用前自动触发

链名映射

```
_CHAIN_MAP = {
    "solana": "solana",
    "bsc": "bsc",
    "base": "base",
    "eth": "eth",
    "ethereum": "eth", # "ethereum"
}
```

Session 复用

```
def _get_session(self) -> aiohttp.ClientSession:
    if self._session is None or self._session.closed:
        self._session = aiohttp.ClientSession()
    return self._session
```

- 惰性创建 aiohttp.ClientSession
- 自动检测关闭状态并重建
- 全局单例 ave = AveClient() 保证进程内只有一个 HTTP 连接池

Data REST 请求格式

```
async def _data_get(self, path: str, params: dict = None) -> Optional[dict]:
    url = f"{AVE_DATA_BASE}{path}" # https://data.ave-api.xyz/v2{path}
```

```

headers = {
  "X-API-KEY": AVE_API_KEY,
  "Content-Type": "application/json",
}
# #####: {"status": 1, "data": {...}, "msg": "..."}
# status == 1 #####
return data.get("data")

```

Trade REST 请求格式

```

async def _trade_post(self, path: str, body: dict) -> Optional[dict]:
  url = f"{AVE_TRADE_BASE}{path}" # https://bot-api.ave.ai{path}
  headers = {
    "AVE-ACCESS-KEY": AVE_API_KEY,
    "Content-Type": "application/json",
  }
  # #####: {"status": 200, "data": {...}, "msg": "..."}
  # status == 200 ■ 1 #####
  return data.get("data")

```

格式转换层

`_convert_risk_to_goplus()` 静态方法将 AVE

安全检测结果转换为内部标准格式，使整个系统无需关心数据源差异：

- `is_honeypot: int(-1/0/1) → bool`
- `has_code: int(0/1) → bool` `is_open_source`
- `holders_detail: array → top10_holder_pct + top1_holder_pct`
- 保留 `_ave_raw` 原始数据，供调试使用

11. AI 决策引擎概览

AI 决策引擎建立在 AVE 数据基础设施之上，将结构化数据转化为交易决策。

三层决策架构

```

##### (### AVE ###)
■
▼
L1 ##### (####)
#####: 9 ##### (>, <, ==, !=, >=, <=, in, not_in, between)
#####: AND / OR #####
### L1 → ### L2
■
▼
L2 ##### (Claude Opus 4.6, ~2■)
### LLM #####: ##### + ##### + #####
###: {action: buy/sell/hold, confidence: 0-100, reasoning: "..."}
confidence >= 70 → ### L3
■
▼

```

```
L3 █████ (3-5 █)
3 █████ (Haiku): ███ / ███ / ██████████
██ (Sonnet): ████████
██ (Sonnet): ████████
███: █████ → ████████
```

三层记忆系统

记忆注入到 L2 决策 Prompt 中，使 AI 能从过去的成功和失败中学习。

Regime 市场状态检测 (7 状态)

风控系统 (15 项检查)

12. 效果指标与运营数据

数据覆盖

性能指标

交易能力

AI 决策质量

13. 链支持矩阵

14. 技术创新点

14.1 基于 AVE 的统一数据层

传统方案需要对接多个独立 API（DexScreener、GoPlus、OKX DEX 等），存在认证碎片化、格式不统一、限速各异等问题。AiTrading Pro 通过 AVE Cloud Skill 实现统一数据层：

- 单一 API Key：一个 AVE Key 覆盖数据查询 + 安全检测 + 交易执行
- 统一格式：ave_client.py 内部做一次格式转换，上层模块完全无感
- 统一限速：1 RPS 限速器统管所有请求，不会因为某个模块超频导致封禁

14.2 AVE 数据驱动的毫秒级打分引擎

传统热币扫描依赖定时批量查询。AiTrading Pro 基于 AVE PriceFeed 实现了事件驱动的实时打分。每 2 秒的价格回调直接触发 HotCoinManager 重新计算 100 分，代币进出榜单延迟从分钟级降至秒级。退出机制（5 规则）同样由价格回调驱动，无需额外轮询。

14.3 PriceFeed 引用计数

创新的引用计数架构解决了代币追踪生命周期管理问题：

- 多个消费者（HotCoinManager / SimTrader / PerformanceTracker）可以独立注册和注销同一代币
- 只要有一个消费者还在关注，价格追踪就继续
- 全部消费者注销后，自动停止轮询，释放 API 配额
- 避免了“退榜后模拟盘无法止损”和“重复订阅浪费配额”两个常见问题

14.4 自托管交易的四步分离架构

AVE chain-wallet API 将交易拆分为 报价 → 构造 → 签名 → 广播 四个独立步骤，其中签名环节完全在本地完成。这种架构既享受了 AVE DEX 聚合的最优路由，又保证了私钥零泄露。CRISIS 模式下滑点自动放宽到 5%，优先保证清仓成交。

14.5 三层供给 + 五维度评估的聪明钱体系

聪明钱地址不是静态列表，而是一个动态生态。AVE 提供基础种子地址，内部 miner 从链上数据（毕业代币早期买家、热币 Top Holders）挖掘新地址，Dune 批量补充。所有地址经 v3 五维度（胜率/PNL/规模/活跃度/时效性）持续评估，实时 bot 检测 + 定期降级/移除，保证库的质量。

14.6 L1-L2-L3 渐进式 AI 决策 + 牛熊辩论

不是所有信号都需要 Claude Opus 级别的分析。L1 规则引擎以毫秒级过滤掉 80% 的低质量信号，L2 快评在 2 秒内给出初步判断，只有高置信度信号才进入 L3 多角色辩论。L3 引入 Bull Agent 和 Bear Agent 对抗式推理，强制考虑反面论据，避免单一模型的确认偏差。置信度评分决定仓位大小：高置信度 → 大仓位，低置信度 → 小仓位或放弃。

15. 环境配置

必需环境变量

```
# AVE Cloud Skill■■■■■■■■■■■■■■■■■■■■■  
AVE_API_KEY=<your-ave-api-key>  
  
# AVE ■■■■■ URL■■■■■■■■■■■■■■■■■■■■■  
# AVE_DATA_BASE=https://data.ave-api.xyz/v2  
# AVE_TRADE_BASE=https://bot-api.ave.ai
```

钱包配置（交易执行需要）

```
# Solana ■■■
SOLANA_WALLET_ADDRESS=<your-solana-address>
SOLANA_PRIVATE_KEY=<your-solana-private-key>

# EVM ■■■BSC/ETH/Base ■■■
EVM_WALLET_ADDRESS=<your-evm-address>
EVM_PRIVATE_KEY=<your-evm-private-key>
```

辅助服务

```
# Helius██████████ Solana DEX ██████████
HELIUS_API_KEY=<your-helius-key>

# Claude AI██████████
ANTHROPIC_API_KEY=<your-anthropic-key>

# Supabase██████████
SUPABASE_URL=<your-supabase-url>
SUPABASE_KEY=<your-supabase-key>

# Dune Analytics██████████
DUNE_API_KEY=<your-dune-key>
```

启动命令

```
cd services/pump-scanner
python main.py

# main.py ██████████:
# - HotCoinManager (AVE trending ███ + █████)
# - PriceFeed (AVE token detail ███)
# - SecurityChecker (AVE contracts ███)
# - SmartMoneyTracker (AVE smart_wallet + █████)
# - TradeExecutor (AVE chainWallet █████)
# - AIDecisionEngine (L1/L2/L3 ███)
# - PumpScanner (pump.fun █████)
# - KOLMonitor (KOL █████)
# - BTCETHAnalyzer (█████)
# - AIOptimizer (█████ Agent)
```

项目仓库结构

```
Agent-Trading/
■■■ services/pump-scanner/ # Python ■■
```

```

■ ■■■ ave_client.py # AVE ■■■■
■ ■■■ hot_coin_manager.py # ■■■■■■
■ ■■■ hot_scorer.py # 12 ■■■■
■ ■■■ price_feed.py # ■■■■■■
■ ■■■ smart_money_tracker.py # ■■■■■■
■ ■■■ collector.py # pump.fun ■■■■
■ ■■■ config.py # ■■■■ AVE ■■■
■ ■■■ main.py # ■■■■
■ ■■■ agent/
■ ■■■ trade_executor.py # ■■■■■■AVE chain-wallet■
■ ■■■ decision_agent.py # L2 AI ■■
■ ■■■ debate.py # L3 ■■■■
■ ■■■ rule_engine.py # L1 ■■■■
■ ■■■ regime_detector.py # 7 ■■■■■■
■ ■■■ risk_manager.py # 15 ■■■■■■
■ ■■■ event_listener.py # EventBus ■■■■
■ ■■■ position_monitor.py # ■■■■ + TP/SL
■ ■■■ memory/
■ ■■■ working_memory.py # ■■■■
■ ■■■ episodic_memory.py# ■■■■
■ ■■■ semantic_memory.py# ■■■■
■■■ apps/app/ # Flutter ■■■■
■■■ apps/portal/ # Next.js ■■■■

```

16. License

MIT License

Copyright (c) 2026 AiTrading Pro

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.